

### Q1: Explain organization of Microprocessor based system.

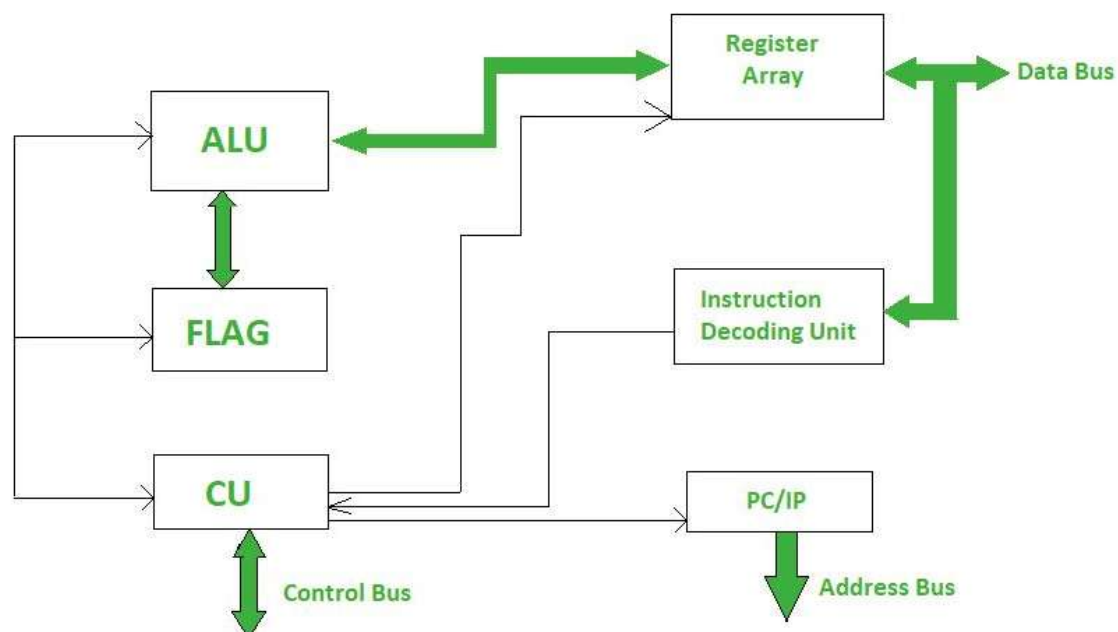
A Microprocessor is a programmable device that takes in input, performs some arithmetic and logical operations over it and produces the desired output. In simple words, a Microprocessor is a digital device on a chip that can fetch instructions from memory, decode and execute them, and give results. It is an important part of a computer architecture without which you will not be able to perform anything on your computer.

#### Block Diagram of a Microprocessor

A Microprocessor takes a bunch of instructions in machine language and executes them, telling the processor what it has to do. The microprocessor performs three basic things while executing the instruction:

- It performs some basic operations like addition, subtraction, multiplication, division, and some logical operations using its Arithmetic and Logical Unit (ALU). New Microprocessors also perform operations on floating-point numbers.
- Data in microprocessors can move from one location to another.
- It has a Program Counter (PC) register that stores the address of the next instruction based on the value of the PC, Microprocessor jumps from one location to another and makes decisions.

A typical Microprocessor structure looks like this.



### **Clock Speed of different Microprocessor:**

- **16-bit Microprocessor**

8086: 4.7MHz, 8MHz, 10MHz

8088: more than 5MHz

80186/80188: 6MHz

80286: 8MHz

- **32-bit Microprocessor**

INTEL 80386: 16MHz to 33MHz

INTEL 80486: 16MHz to 100MHz

PENTIUM: 66MHz

- **64-bit Microprocessor**

INTEL CORE-2: 1.2GHz to 3GHz

INTEL i7: 2.66GHz to 3.33GHz

INTEL i5: 2.4GHz to 3.6GHz

INTEL i3: 2.93GHz to 3.33GHz

We do not have any 128-bit Microprocessor at work at present one of the reasons for this is that we are a long way from exhausting the 64-bit address space itself, we use it at a constant rate of roughly 2 bits every 3 years. At present we have only used 48 bits of 64 bits so why require 128-bit address space. Also, 128-bit Microprocessor would be much slower than the 64 bit Microprocessor.

### **Types of Processor**

#### **Complex Instruction Set Computer (CISC)**

CISC or Complex Instruction Set Computer is a computer architecture where instructions are such that a single instruction can execute multiple low-level operations like loading from memory, storing into memory, or an arithmetic operation, etc. It has multiple addressing nodes within a single instruction. CISC makes use of very few registers.

#### **Example of CISC are**

- Intel 386
- Intel 486
- Pentium

- Pentium Pro
- Pentium II
- Pentium III
- Motorola 68000
- Motorola 68020
- Motorola 68040 etc.

### **Reduced Instruction Set Computer (RISC)**

RISC or Reduced Instruction Set Computer is a computer architecture where instruction is simple and designed to get executed quickly. Instructions get completed in one clock cycle this is because of the optimization of instructions and pipelining (a technique that allows for simultaneous execution of parts, or stages, of instructions more efficiently process instructions). RISC makes use of multiple registers to avoid large interactions with memory. It has few addressing nodes.

#### **Example of RISC are**

- IBM RS6000
- MC88100
- DEC Alpha 21064
- DEC Alpha 21164
- DEC Alpha 21264

### **Explicitly Parallel Instruction Computing (EPIC)**

EPIC or Explicitly Parallel Instruction Computing permits computers to execute instructions parallel using compilers. It allows complex instructions execution without using higher clock frequencies. EPIC encodes its instruction into 128-bit bundles. each bundle contains three instructions which are encoded in 41 bits each and a 5-bit template field (contains information about types of instructions in a bundle and which instructions can be executed in parallel).

#### **Example of EPIC is**

- IA-64 (Intel Architecture-64)

## Q2. Explain Buses of Microprocessor.

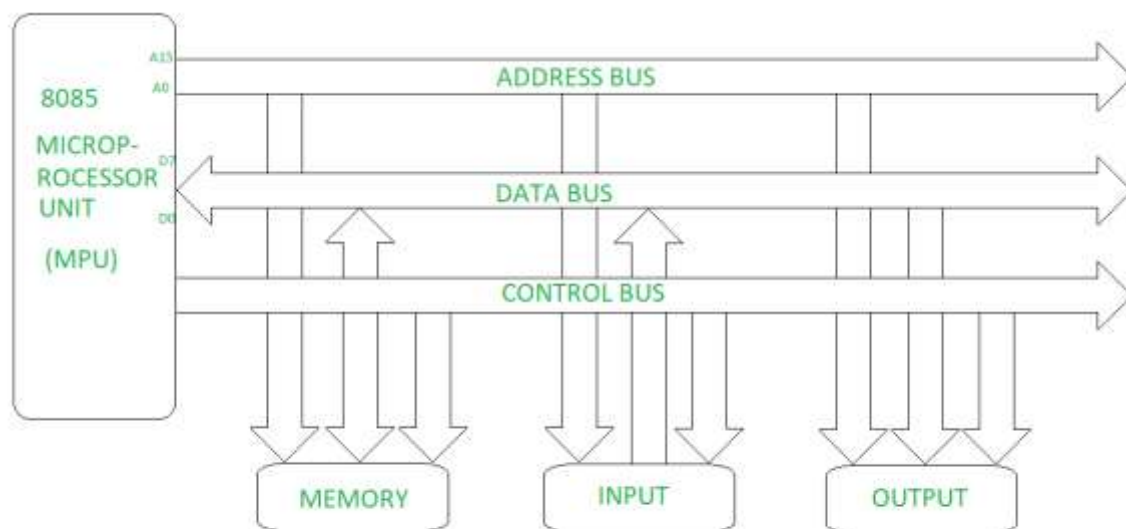
The bus organization of the 8085 microprocessor is the way in which the microprocessor communicates with other devices in a computer system. The 8085 microprocessor has a 16-bit address bus, an 8-bit data bus, and various control signals that are used to manage data transfer and other operations.

The address bus is used to specify the memory location or device with which the microprocessor wants to communicate. It is 16 bits wide, which allows the microprocessor to address up to 64K bytes of memory. The address bus is unidirectional, which means that data can only flow in one direction from the microprocessor to the addressed device.

The data bus is used to transfer data between the microprocessor and other devices. It is 8 bits wide, which means that data can be transferred in byte-sized chunks. The data bus is bidirectional, which means that data can flow in either direction between the microprocessor and other devices.

In addition to the address and data buses, the 8085 microprocessor has various control signals that are used to manage data transfer and other operations. These control signals include the read (RD), write (WR), and hold (HLDA) signals, among others. The RD and WR signals are used to control data transfer to and from memory or other devices, while the HLDA signal is used to indicate that the microprocessor is in a hold state and cannot execute instructions.

Bus is a group of conducting wires which carries information, all the peripherals are connected to microprocessor through Bus. Diagram to represent bus organization system of 8085 Microprocessor.



Bus organization system of 8085 Microprocessor

## **Why use Bus organization in 8085 microprocessor ?**

There are several reasons why bus organization is used in the 8085 microprocessor:

1. **Memory access:** The bus organization is used for accessing memory by transferring the address of the memory location through the address bus and the data to be stored or retrieved through the data bus. This enables the microprocessor to read and write data to and from memory, which is essential for executing instructions and storing data.
2. **I/O operations:** The bus organization is used for performing input/output (I/O) operations by transferring the input/output device address through the address bus and the data to be input or output through the data bus. This enables the microprocessor to communicate with peripheral devices such as keyboards, displays, and sensors.
3. **Interrupt handling:** The bus organization is used for interrupt handling, where the microprocessor uses the address bus to fetch the interrupt vector and the data bus to fetch the interrupt service routine. This enables the microprocessor to respond to external events and perform time-critical operations.
4. **DMA operations:** The bus organization is used for performing Direct Memory Access (DMA) operations, where the data transfer between the memory and I/O devices takes place without the intervention of the microprocessor. This enables high-speed data transfer between devices and reduces the load on the microprocessor.
5. **Control signal transfer:** The bus organization is used for transferring control signals between the microprocessor and other components of the system. This enables the microprocessor to control the operation of devices and coordinate the execution of instructions.

### **There are three types of buses.**

- **Address bus -**

The address bus is a unidirectional bus that is used to carry the memory or I/O device address to which the data is to be transferred. The address bus in the 8085 microprocessor is 16-bit wide.

It is a group of conducting wires which carries address only. Address bus is unidirectional because data flow in one direction, from microprocessor to memory or from microprocessor to Input/output devices (That is, Out of Microprocessor). Length of Address Bus of 8085 microprocessor is 16 Bit (That is, Four Hexadecimal Digits), ranging from 0000 H to FFFF H, (H denotes Hexadecimal). The microprocessor 8085 can transfer maximum 16 bit address which means it can address 65, 536 different memory location. The Length of the address bus determines the amount of memory a system can address. Such as a system with a 32-bit address bus can address  $2^{32}$  memory locations. If each memory location holds one byte, the

addressable memory space is 4 GB. However, the actual amount of memory that can be accessed is usually much less than this theoretical limit due to chipset and motherboard limitations.

- **Data bus -**

The data bus is an 8-bit bidirectional bus that is used to transfer data between the microprocessor and other components such as memory and I/O devices. It is used to carry data to or from the memory or input/output devices.

It is a group of conducting wires which carries Data only. Data bus is bidirectional because data flow in both directions, from microprocessor to memory or Input/Output devices and from memory or Input/Output devices to microprocessor. Length of Data Bus of 8085 microprocessor is 8 Bit (That is, two Hexadecimal Digits), ranging from 00 H to FF H. (H denotes Hexadecimal). When it is write operation, the processor will put the data (to be written) on the data bus, when it is read operation, the memory controller will get the data from specific memory block and put it into the data bus. The width of the data bus is directly related to the largest number that the bus can carry, such as an 8 bit bus can represent  $2^8$  unique values, this equates to the number 0 to 255. A 16 bit bus can carry 0 to 65535.

- **Control bus -**

The control bus is a bidirectional bus that is used to carry control signals between the microprocessor and other components such as memory and I/O devices. It is used to transmit commands to the memory or I/O devices for performing specific operations.

It is a group of conducting wires, which is used to generate timing and control signals to control all the associated peripherals, microprocessor uses control bus to process data, that is what to do with selected memory location. Some control signals are:

Memory read

Memory write

1. I/O read
2. I/O Write
3. Opcode fetch

### **Uses of Bus organization in 8085 microprocessor :**

Some of the important uses of bus organization in the 8085 microprocessor are:

1. **Memory access:** The bus organization is used for accessing the memory by transferring the address of the memory location through the address bus and the data to be stored or retrieved through the data bus.

2. I/O operations: The bus organization is used for performing input/output operations by transferring the input/output device address through the address bus and the data to be input or output through the data bus.
3. Interrupt handling: The bus organization is used for interrupt handling, where the microprocessor uses the address bus to fetch the interrupt vector and the data bus to fetch the interrupt service routine.
4. DMA operations: The bus organization is used for performing Direct Memory Access (DMA) operations, where the data transfer between the memory and I/O devices takes place without the intervention of the microprocessor.

#### **Advantages:**

**Flexibility:** The bus organization used in the 8085 microprocessor allows it to communicate with a wide range of devices. This flexibility makes it well-suited for use in a variety of computer systems, including embedded systems, personal computers, and other devices.

**Modularity:** The bus organization makes it easy to add or remove devices from a computer system. This modularity allows system designers to customize the system to meet the needs of specific applications.

**Scalability:** The bus organization used in the 8085 microprocessor is scalable, which means that it can be used in systems of varying sizes and complexity. This scalability makes it well-suited for use in systems that require a wide range of performance levels.

**Low Cost:** The bus organization used in the 8085 microprocessor is relatively simple and inexpensive to implement. This makes it an attractive option for low-cost, embedded applications.

#### **Disadvantages:**

**Limited Bandwidth:** The bus organization used in the 8085 microprocessor has a limited bandwidth, which can limit the performance of the processor in high-performance applications.

**Latency:** The bus organization can introduce latency, which is the delay between the time a command is issued and the time the response is received. This latency can be a problem in real-time applications that require immediate responses.

**Data Integrity:** The bus organization used in the 8085 microprocessor is vulnerable to data corruption due to electromagnetic interference and other sources of noise. This can lead to errors in data transmission and processing.

**Complexity:** The bus organization used in the 8085 microprocessor can be complex to implement and troubleshoot, which can increase the cost and time required to develop and maintain computer systems.

### Q3. Define Microprocessor and explain difference between microprocessor and microcontroller.

A **microprocessor** is a CPU on a single chip, requiring external memory and peripherals, making it powerful for general computing like PCs; a **microcontroller** integrates a CPU, memory, and I/O ports all on one chip, creating a mini-computer ideal for specific, low-power embedded tasks like in home appliances. The main difference lies in integration: microprocessors are core processors needing support chips, while microcontrollers are self-contained systems for dedicated control functions.

#### Microprocessor Definition

A microprocessor is a single integrated circuit (IC) that functions as the Central Processing Unit (CPU) of a computer, executing instructions and performing arithmetic and logic operations, but needs external RAM, ROM, and I/O devices to form a complete system.

#### Microprocessor vs. Microcontroller Comparison

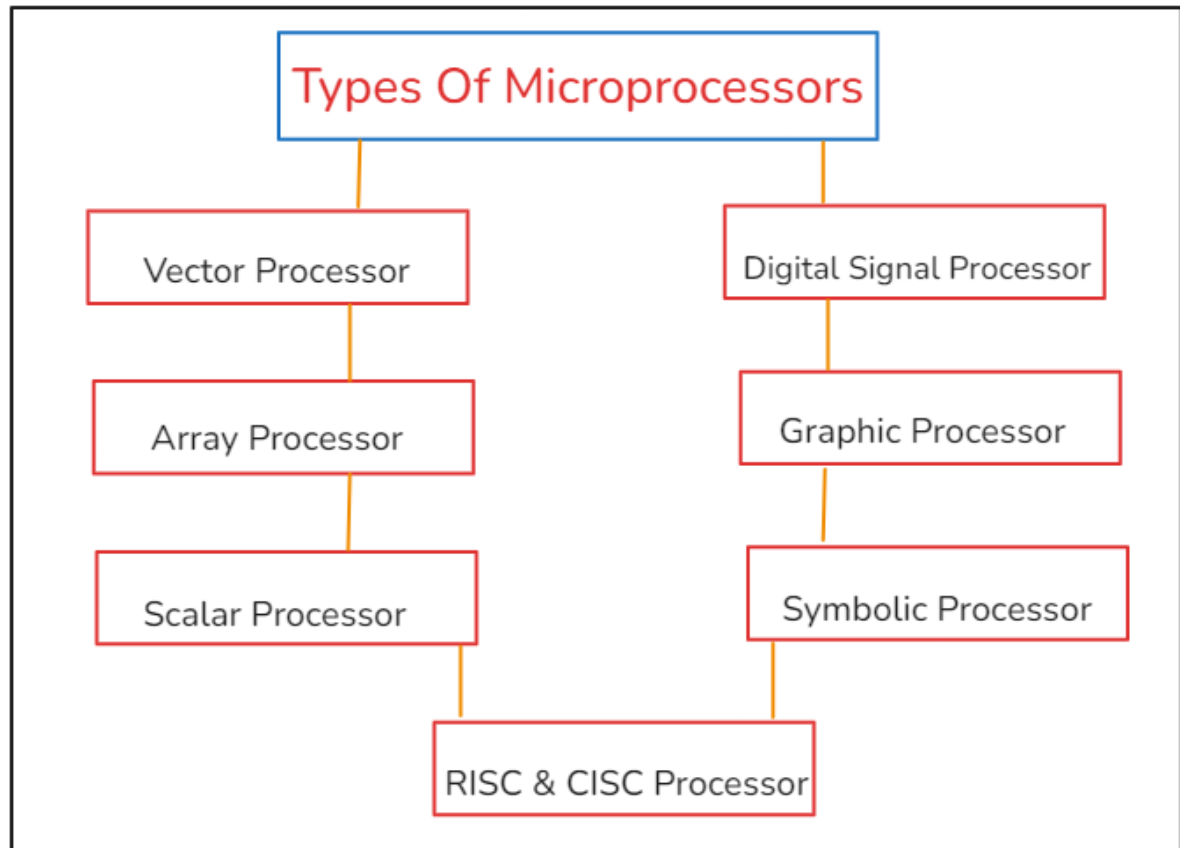
| Feature         | Microprocessor (MPU)                                   | Microcontroller (MCU)                                                     |
|-----------------|--------------------------------------------------------|---------------------------------------------------------------------------|
| Integration     | Only CPU; needs external memory, I/O.                  | CPU, RAM, ROM, I/O all on one chip (a mini-computer).                     |
| Application     | General-purpose computing (PCs, laptops, smartphones). | Specific, embedded tasks (washing machines, remote controls, automotive). |
| Architecture    | Often Von Neumann (shared bus for data/instructions).  | Often Harvard (separate buses for data/instructions).                     |
| Cost/Complexity | More expensive, complex, high performance.             | Inexpensive, simpler, lower power.                                        |
| Power           | Higher power consumption.                              | Low power, good for battery devices, power-saving modes.                  |
| Bus             | Uses external bus for interfacing.                     | Uses internal controlling bus.                                            |
| Flexibility     | High; easy to upgrade components like RAM.             | Low; fixed internal resources.                                            |

#### Key Takeaway

- **Microprocessor (MPU):** The "brain" for complex, versatile computing, requiring many external components.
- **Microcontroller (MCU):** A self-sufficient "mini-computer" for dedicated, efficient control in embedded devices.

#### Q4. Classification of Microprocessor based on size, application and speed.

##### Types of Microprocessors



##### Vector Processor

A vector processor is a type of central processing unit (CPU) that is designed to perform mathematical operations on arrays of data, called vectors, more efficiently than a scalar processor, which operates on single data elements. Vector processors can perform operations on multiple data elements simultaneously, which can lead to faster and more efficient processing. Vector processors can be found in some supercomputers and servers, as well as in some specialized graphics processing units (GPUs).

##### Array Processor or SIMD Processor

Array processors are also designed for vector computations. The difference between an array processor and a scalar processor is that a vector processor uses multiple vector pipelines whereas an array processor employs a number of processing elements to operate in parallel. Array processors can also be found in some supercomputers and servers, as well as in some specialized graphics processing units (GPUs). Array processors are different from scalar processors, which operate on single data elements and are more common in general-purpose computing applications.

## **Scalar Processor**

It is a processor which executes scalar data. The simplest scalar processor makes the processing of only integer instruction using fixed point operands. A powerful scalar processor makes the processing of both integers as well as floating point numbers. It contains an integer ALU and a floating point unit (FPU) on the same CPU chip. A scalar processor may be a CISC or RISC processor. A super scalar processor contains multiple pipelines and executes more than one instruction per clock cycle.

## **RISC and CISC Processor**

RISC (Reduced Instruction Set Computing) and CISC (Complex Instruction Set Computing) are two approaches to designing a central processing unit (CPU), which is the main component of a computer that performs most of the processing tasks. RISC processors have a smaller, simpler instruction set, which means they have fewer types of instructions that they can execute. This makes them easier to design and manufacture, and allows them to execute instructions faster than CISC processors. RISC processors are typically used in devices that require high performance and low power consumption, such as smartphones and tablets. RISC processors are faster and more efficient than CISC processors.

CISC processors, on the other hand, have a larger and more complex instruction set, which means they can execute a wider range of instructions. This makes them more versatile, but also more expensive and slower to execute instructions than RISC processors. CISC processors are typically used in devices that require more flexibility, such as desktop computers and servers.

For more detail please refer to RISC and CISC articles.

## **Digital Signal Processor**

A digital signal processor (DSP) is a specialized microprocessor designed to perform high-speed mathematical operations on digital signals, which are sequences of digital data representing physical quantities such as sound, images, and other sensory information. DSPs are used in a range of applications that require fast processing of digital signals, including audio and video processing, telecommunications, and scientific computing. DSPs are optimized for performing mathematical operations common in signal processing, like filtering, convolution, and FFTs, and have specialized instructions and hardware support for these tasks, making them faster and more efficient than general-purpose microprocessors.

## **Symbolic Processor**

It is a processor that is designed to perform symbolic computation, which is the manipulation of mathematical expressions and symbols using algorithms and rules of symbolic logic. Such processors are designed for expert systems, machine intelligence,

knowledge-based system, pattern recognition, etc. Symbolic processors are also called PROLOG processors.

### **Graphics Processor**

A graphics processor is a specialized microprocessor designed to accelerate the rendering of 2D and 3D graphics in a computer system. It is used in devices like PCs, laptops, smartphones, and gaming consoles, as well as specialized graphics hardware. A GPU(Graphics processing unit) is different from a CPU and is specifically designed to perform complex calculations for high-quality graphics in real time. It performs tasks related to graphics renderings, like shading and texturing, more efficiently than a CPU. A GPU is also used in specialized graphics hardware such as graphics cards and accelerators.

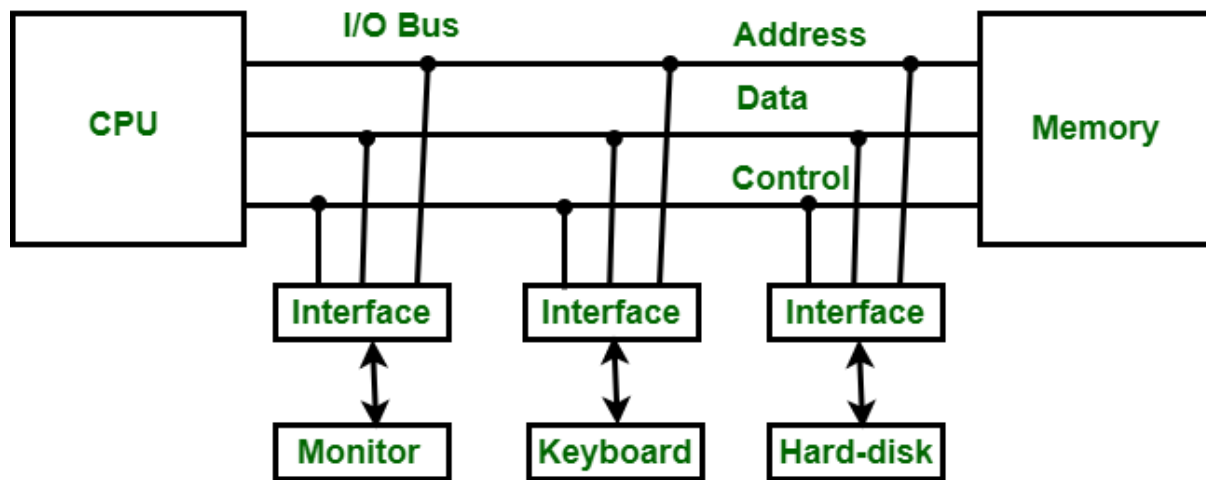
### **Applications of Microprocessor**

- **Computer:** The Microprocessors are the CPU's in computers.
- **Embedded Systems:** It is utilized as the main processing block in the Embedded systems such as Washing machines, microwaves and other equipment's.
- **Industrial Automation:** It can be used to control Industrial machinery and equipment's.
- **Automotive:** The Modern day Vehicles uses Microprocessors in there ECUs.
- **Telecommunications:** The Microprocessors are utilized in the Telecommunications systems such as routers, modems, and switches.

### **Q5. Explain languages for communication between microprocessor to other peripheral and vice-versa.**

Input-Output Interface is used as a method which helps in transferring information between the internal storage devices, i.e., memory, and the external peripheral device. A peripheral device provides input and output for the computer; it is also called an Input-Output device.

For Example, A keyboard and mouse, which provide input to the computer, are called input devices while a monitor and printer provide output to the computer, are called output devices. Just like the external hard drives, there is also availability of some peripheral devices which are able to provide both input and output.



#### Input-Output Interface

In micro-computer base system, the only purpose of peripheral devices is just to provide **special communication links** for the interfacing them with the CPU. To resolve the differences between peripheral devices and CPU, there is a special need for communication links.

The major differences are as follows:

1. The nature of peripheral devices is electromagnetic and electro-mechanical. The nature of the CPU is electronic. There is a lot of difference in the mode of operation of both peripheral devices and CPU.
2. There is also a synchronization mechanism because the data transfer rate of peripheral devices are slow than CPU.
3. In peripheral devices, data code and formats are differ from the format in the CPU and memory.
4. The operating mode of peripheral devices are different and each may be controlled so as not to disturb the operation of other peripheral devices connected to CPU.

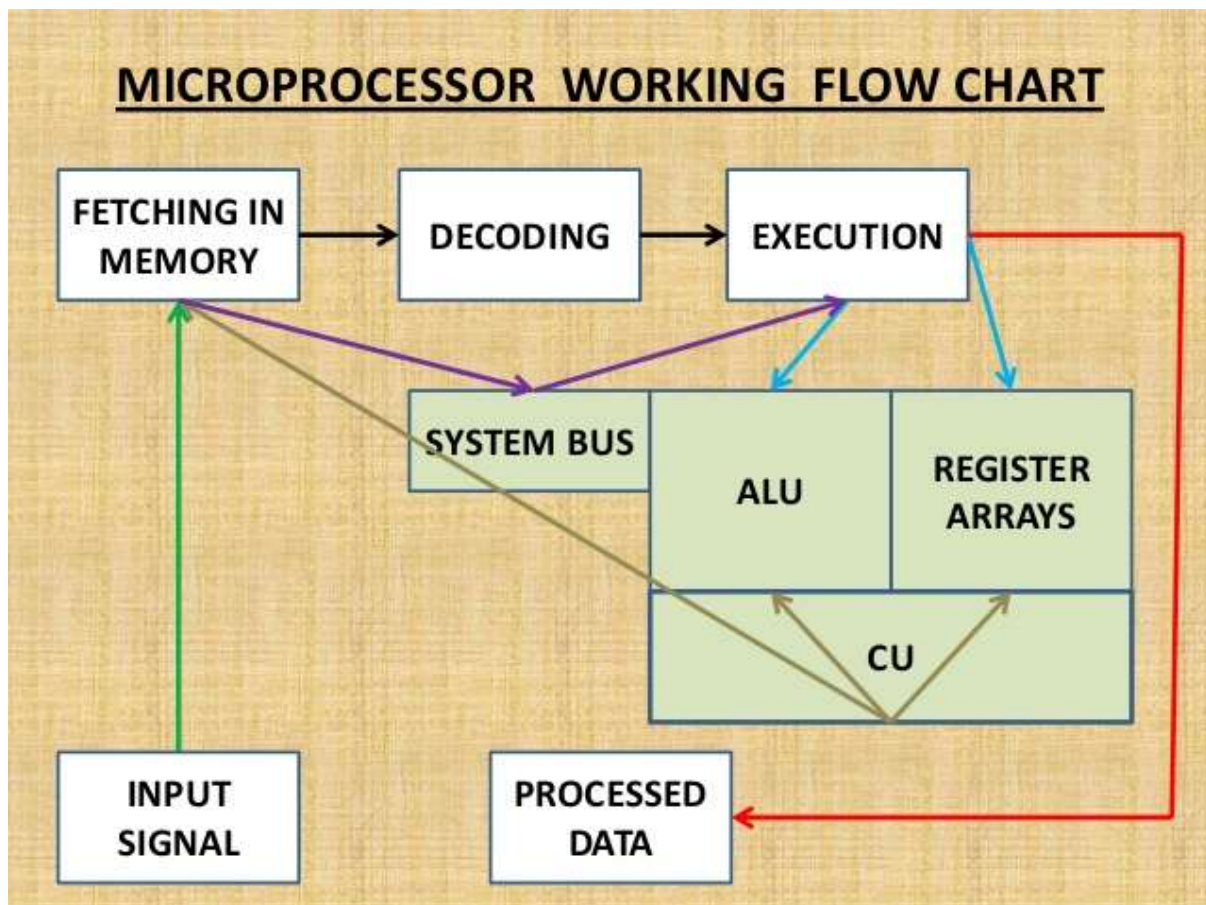
There is a special need of the additional hardware to resolve the differences between CPU and peripheral devices to supervise and synchronize all input and output devices.

#### Functions of Input-Output Interface:

1. It is used to synchronize the operating speed of CPU with respect to input-output devices.
2. It selects the input-output device which is appropriate for the interpretation of the input-output signal.
3. It is capable of providing signals like control and timing signals.
4. In this data buffering can be possible through data bus.

5. There are various error detectors.
6. It converts serial data into parallel data and vice-versa.
7. It also convert digital data into analog signal and vice-versa.

**Q6. Briefly explain how does the microprocessor work?**



A microprocessor works by repeatedly performing the Fetch-Decode-Execute cycle, fetching instructions and data from memory, interpreting them (decoding), performing operations (like math or logic in the ALU), and storing the results, all synchronized by a clock signal, to run programs on a silicon chip. It uses internal registers for temporary storage and control units to manage the flow, acting as the brain of a computer by executing binary commands.

### Core Components

- **Arithmetic Logic Unit (ALU):** Performs mathematical (add, subtract) and logical (AND, OR) operations.
- **Control Unit:** Directs and synchronizes all operations, sending timing signals.

- **Registers:** Small, fast memory locations for temporarily holding data and instructions (e.g., Program Counter).

### The Fetch-Decode-Execute Cycle

#### 1. Fetch:

The Control Unit gets the next instruction and data from memory (RAM) using the address bus.

#### 2. Decode:

The instruction is interpreted by the Control Unit to understand what operation to perform.

#### 3. Execute:

The ALU performs the operation (e.g., adds numbers), using data from registers or memory.

#### 4. Store (Writeback):

The result is written back to a register or memory.

### Key Concepts

- **Clock Speed:** Measured in GHz, it determines how many cycles (fetches, decodes, etc.) the processor can perform per second, dictating its speed.
- **Binary Data:** The microprocessor processes data as binary 0s and 1s.
- **Buses:** Data buses move data, while address buses carry memory locations between the processor and memory.

### Q7. Explain in brief programming model of 8085 microprocessor?

A microprocessor is fabricated on a single integrated circuit (IC) or chip that is used as a central processing unit (CPU). The 8085 microprocessor is an 8-bit microprocessor that was developed by **Intel in the mid-1970s**. It was widely used in the early days of personal computing and was a popular choice for enthusiasts due to its simplicity and ease of use. The architecture of the 8085 microprocessor consists of several key components, including the accumulator, registers, program counter, stack pointer, instruction register, flags register, data bus, address bus, and control bus.

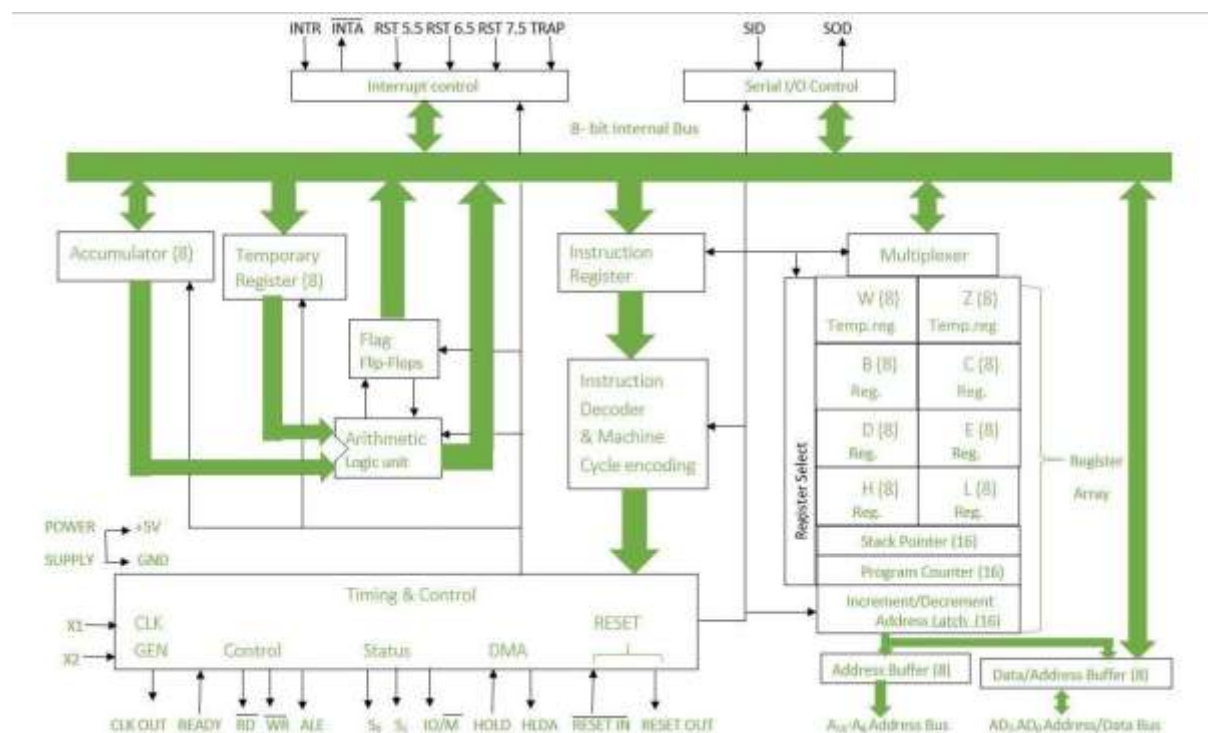
The accumulator is an 8-bit register that is used to store arithmetic and logical results. It is the most commonly used register in the 8085 microprocessor and is used to perform arithmetic and logical operations such as addition, subtraction, and bitwise operations.

### Features of 8085 Microprocessor

- The 8085 microprocessor has six general-purpose registers (B, C, D, E, H, and L), which can be paired as BC, DE, and HL for 16-bit operations.

- The program counter (PC) is a 16-bit register that holds the address of the next instruction and increments after each instruction execution.
- The stack pointer (SP) is a 16-bit register that keeps track of the top of the stack, used for storing return addresses and temporary data.
- The instruction register is an 8-bit register that holds the instruction currently being executed.
- The flags register is an 8-bit register that stores status flags, including Carry (set when an arithmetic operation generates a carry), Zero (set when the result is zero), Sign (set when the result is negative), and Parity (set when the result has an even number of 1s).
- The data bus is an 8-bit bidirectional bus that transfers data between the microprocessor and memory or other devices.
- The address bus is a 16-bit unidirectional bus that specifies memory locations and devices the microprocessor accesses.
- The control bus consists of signals that manage microprocessor operations, including Read (to fetch data), Write (to store data), Interrupt (to signal an external event), and Reset (to restart the microprocessor).

8085 is an 8-bit, general-purpose microprocessor. It consists of the following functional units:



## **Arithmetic and Logic Unit (ALU)**

It is used to perform mathematical operations like addition, multiplication, subtraction, division, decrement, increment, etc. Different operations are carried out in ALU: *Logical operations, Bit-Shifting Operations, and Arithmetic Operations*.

## **Flag Register**

It is an 8-bit register that stores either 0 or 1 depending upon which value is stored in the accumulator. Flag Register contains 8-bit out of which 5-bits are important and the rest of 3-bits are "don't Care conditions". The flag register is a dynamic register because after each operation to check whether the result is zero, positive or negative, whether there is any overflow occurred or not, or for comparison of two 8-bit numbers carry flag is checked. So for numerous operations to check the contents of the accumulator and from that contents if we want to check the behavior of given result then we can use Flag register to verify and check. So we can say that the flag register is a status register and it is used to check the status of the current operation which is being carried out by ALU.

## **Different Fields of Flag Register**

### **Carry Flag**

1. **Parity Flag**
2. **Auxiliary Carry Flag**
3. **Zero Flag**
4. **Sign Flag**

## **Accumulator**

Accumulator is used to perform I/O, arithmetic, and logical operations. It is connected to ALU and the internal data bus. The accumulator is the heart of the microprocessor because for all arithmetic operations Accumulators' 8-bit pin will always be connected with ALU and in most-of times all the operations carried by different instructions will be stored in the accumulator after operation performance.

## **General Purpose Registers**

There are six general-purpose registers. These registers can hold 8-bit values. These 8-bit registers are B,C,D,E,H,L. These registers work as 16-bit registers when they work in pairs like B-C, D-E, and H-L. Here registers W and Z are reserved registers. We can't use these registers in arithmetic operations. It is reserved for microprocessors for internal operations like swapping two 16-bit numbers. We know that to swap two numbers we need a third variable hence here W-Z register pair works as temporary registers and we can swap two 16-bit numbers using this pair.

## Program Counter

Program Counter holds the address value of the memory to the next instruction that is to be executed. It is a 16-bit register.

**For Example:** Suppose current value of Program Counter :  $[PC] = 4000H$

*(It means that next executing instruction is at location 4000H. After fetching, program Counter(PC) always increments*

*by +1 for fetching of next instruction.)*

## Stack Pointer

It works like a stack. In stack, the content of the register is stored that is later used in the program. It is a 16-bit special register. The stack pointer is part of memory but it is part of Stack operations, unlike random memory access. Stack pointer works in a continuous and contiguous part of the memory, whereas Program Counter(PC) works in random memory locations. This pointer is very useful in stack-related operations like **PUSH, POP, and nested CALL requests** initiated by Microprocessor. *It reserves the address of the most recent stack entry.*

## Temporary Register

It is an 8-bit register that holds data values during arithmetic and logical operations.

## Instruction register and decoder

It is an 8-bit register that holds the instruction code that is being decoded. The instruction is fetched from the memory.

## Timing and Control Unit

The timing and control unit comes under the CPU section, and it controls the flow of data from the CPU to other devices. It is also used to control the operations performed by the microprocessor and the devices connected to it. There are certain timing and control signals like Control signals, [DMA](#) Signals, RESET signals and Status signals.

## Interrupt Control

Whenever a microprocessor is executing the main program and if suddenly an interrupt occurs, the microprocessor shifts the control from the main program to process the incoming request. After the request is completed, the control goes back to the main program. There are 5 interrupt signals in 8085 microprocessors: INTR, TRAP, RST 7.5, RST 6.5, and RST 5.5.

**Priorities of Interrupts:** TRAP > RST 7.5 > RST 6.5 > RST 5.5 > INTR

## Address Bus and Data Bus

The data bus is bidirectional and carries the data which is to be stored. The address bus is unidirectional and carries the location where data is to be stored.

In the 8085 microprocessor, the address bus and data bus are two separate buses that are used for communication between the microprocessor and external devices.

The Address bus is used to transfer the memory address of the data that needs to be read or written. The address bus is a 16-bit bus, allowing the 8085 to access up to 65,536 memory locations.

The Data bus is used to transfer data between the microprocessor and external devices such as memory and I/O devices. The data bus is an 8-bit bus, allowing the 8085 to transfer 8-bit data at a time. The data bus can also be used for instruction fetch operations, where the microprocessor fetches the instruction code from memory and decodes it.

The combination of the address bus and data bus allows the 8085 to communicate with and control external devices, allowing it to execute its program and perform various operations.

### **Serial Input/Output Control**

It controls the serial data communication by using Serial input data and Serial output data.

Serial Input/Output control in the 8085 microprocessor refers to the communication of data between the microprocessor and external devices in a serial manner, i.e., one bit at a time. The 8085 has a serial I/O port (SID/SOD) for serial communication. The SID pin is used for serial input and the SOD pin is used for serial output. The timing and control of serial communication is managed by the 8085's internal circuitry. The 8085 also has two special purpose registers, the Serial Control Register (SC) and the Serial Shift Register (SS), which are used to control and monitor the serial communication.

### **The flow of an Instruction Cycle in 8085 Architecture**

- Execution starts with Program Counter. It starts program execution with the next address field. it fetches an instruction from the memory location pointed by [Program Counter](#).
- For address fetching from the memory, multiplexed address/data bus acts as an address bus and after fetching instruction this address bus will now acts as a data bus and extract data from the specified memory location and send this data on an 8-bit internal bus. For multiplexed address/data bus Address Latch Enable([ALE](#)) Pin is used. If **ALE = 1 (Multiplexed bus is Address Bus otherwise it acts as Data Bus)**.
- After data fetching data will go into the Instruction Register it will store data fetched from memory and now data is ready for decoding so for this Instruction decoder register is used.

- After that timing and control signal circuit comes into the picture. *It sends control signals all over the microprocessor to tell the microprocessor whether the given instruction is for READ/WRITE and whether it is for MEMORY/I-O Device activity.*
- Hence according to timing and control signal pins, logical and arithmetic operations are performed and according to that data fetching from the different registers is done by a microprocessor, and mathematical operation is carried out by ALU. And according to operations Flag register changes dynamically.
- With the help of Serial I/O data pin(SID or SOD Pins) we can send or receive input/output to external devices .in this way execution cycle is carried out.
- *While execution is going on if there is any [interrupt](#) detected then it will stop execution of the current process and Invoke Interrupt Service Routine (ISR) Function. Which will stop the current execution and do execution of the current occurred interrupt after that normal execution will be performed.*

### Uses of 8085 Microprocessor

The 8085 microprocessor is a versatile 8-bit microprocessor that has been used in a wide variety of applications, including:

- **Embedded Systems:** The 8085 microprocessor is commonly used in [embedded systems](#), such as industrial [control systems](#), automotive electronics, and medical equipment.
- **Computer Peripherals:** The 8085 microprocessor has been used in a variety of computer peripherals, such as printers, scanners, and disk drives.
- **Communication Systems:** The 8085 microprocessor has been used in communication systems, such as modems and network interface cards.
- **Instrumentation and Control Systems:** The 8085 microprocessor is commonly used in instrumentation and control systems, such as temperature and pressure controllers.
- **Home Appliances:** The 8085 microprocessor is used in various home appliances, such as washing machines, refrigerators, and microwave ovens.
- **Educational Purposes:** The 8085 microprocessor is also used for educational purposes, as it is an inexpensive and easily accessible microprocessor that is widely used in universities and technical schools.

### Issues in 8085 Microprocessor

Here are some common issues with the 8085 microprocessor:

- **Overheating:** The [8085 microprocessor](#) can overheat if it is used for extended periods or if it is not cooled properly. Overheating can cause the microprocessor to malfunction or fail.
- **Power Supply Issues:** The 8085 microprocessor requires a stable power supply for proper operation. Power supply issues such as voltage fluctuations, spikes, or drops can cause the [microprocessor](#) to malfunction.
- **Timing Issues:** The 8085 microprocessor relies on accurate timing signals for proper operation. Timing issues such as clock signal instability, noise, or interference can cause the microprocessor to malfunction.
- **Memory Interface Issues:** The 8085 microprocessor communicates with memory through its address and data buses. Memory interface issues such as faulty memory chips, loose connections, or address decoding errors can cause the microprocessor to malfunction.
- **Hardware Interface Issues:** The 8085 microprocessor communicates with other devices through its input/output ports. Hardware interface issues such as faulty devices, incorrect wiring, or improper device selection can cause the microprocessor to malfunction.
- **Programming Issues:** The 8085 microprocessor is programmed with machine language or assembly language instructions. Programming issues such as syntax errors, logic errors, or incorrect instruction sequences can cause the microprocessor to malfunction or produce incorrect results.
- **Research and Development:** The 8085 microprocessor is often used in research and development projects, where it can be used to develop and test new digital electronics and computer systems. Researchers and developers can use the microprocessor to prototype new systems and test their performance.
- **Retro Computing:** The 8085 microprocessor is still used by enthusiasts today for retro computing projects. Retro computing involves using older computer systems and technologies to explore the history of computing and gain a deeper understanding of how modern computing systems have evolved.

**Q8. Write basic features of 8085 microprocessor. (clock, data bus , address bus, memory, Number of instruction of execution per second, how many instruction for operation)**

The 8085 is an 8-bit microprocessor with a 16-bit address bus (64KB memory), 8-bit data bus, typically a 3-5 MHz clock, and around 246 instructions, requiring multiple clock cycles (T-states) per instruction for fetching, decoding, and execution, varying by instruction complexity and clock speed. Key features include N-MOS tech, 40 pins, +5V power, and interrupt support.

## Basic Features of 8085 Microprocessor

- **Data Bus:** 8-bit (bidirectional)
- **Address Bus:** 16-bit (unidirectional, A0-A15), allowing addressing of  $2^{16} = 65,536$  (64KB) memory locations.
- **Clock Speed:** Up to 3 MHz or 5 MHz, depending on the variant, using a single +5V supply
- **Memory:** Can access 64KB.
- **Instructions:** Around 246 instructions, each represented by an 8-bit opcode.
- **Registers:** Includes an Accumulator, six 8-bit general-purpose registers (B, C, D, E, H, L) that form pairs, a 16-bit Program Counter (PC), and a 16-bit Stack Pointer (SP).
- **Technology:** N-MOS technology.
- **Pins:** 40-pin Dual In-line Package (DIP).
- **Interrupts:** Supports external interrupts.

## Instruction Execution

- **T-States:** Instructions take multiple clock cycles (T-states) to execute (Fetch + Decode + Execute).
- **Instruction Cycle:** The process of fetching, decoding, and executing one instruction.
- **Machine Cycle:** The basic step for memory/IO access (Fetch, Decode, Execute, Store).

## Performance (Instructions per Second)

- This isn't a fixed number; it depends heavily on the instruction mix and clock frequency (e.g., a 3 MHz clock has a 333ns period). Complex instructions take more T-states.
- For example, at 3 MHz, if an instruction takes 4 T-states (12 clock cycles), it's roughly  $3\text{M} / 12 = 250,000$  instructions/second, but this varies greatly.

## QUESTION-09

### (A) Define: bit, nibble, byte, word and instruction

#### Bit

- **Definition:** The fundamental unit of digital information, representing a single binary value: 0 (off) or 1 (on).
- **Analogy:** Like a single light switch.

#### Nibble

- **Definition:** A group of 4 bits, also called a half-byte or tetrad, capable of representing 16 different values (0-15).
- **Use:** Often used to represent a single hexadecimal digit.

#### Byte

- **Definition:** A group of 8 bits, the most common unit for measuring data size and representing characters (letters, numbers, symbols).
- **Analogy:** A single character like 'A' or '7'.

#### Word

- **Definition:** The natural data size a processor handles, varying by architecture (e.g., 16-bit, 32-bit, 64-bit), often comprising 2, 4, or 8 bytes.
- **Use:** Represents the size of registers or data processed at once.

#### Instruction

- **Definition:** A command in machine language that a computer's CPU executes, often represented by binary patterns (opcodes and operands).
- **Relation:** Instructions and data are both stored as bits, bytes, and words in memory, with the CPU fetching and interpreting them to perform tasks.

### (B) What is meaning by 8-bit processor?

An 8-bit processor is a CPU that processes data in chunks of 8 bits (a byte) at a time, using 8-bit registers and data buses, limiting its direct numerical range to 0-255 but making it ideal for simple, low-power tasks like early gaming consoles and appliances. It defines a generation of computers known for processing data in 8-bit units, contrasting with modern processors that handle much larger bit sizes for greater speed and complexity.

### Key characteristics:

- **Data Width:** Operates on 8 bits of data per clock cycle.
- **Numerical Range:** Can handle values from 0 to 255 ( $2^8$ ) directly.
- **Architecture:** Features 8-bit registers, Arithmetic Logic Unit (ALU), and data paths.
- **Memory:** Typically addresses limited memory, often up to 64KB (65,536 bytes) using 16-bit address registers.
- **Applications:** Suited for simpler tasks in toys, remote controls, and basic embedded systems due to lower power and cost.
- **Examples:** [Intel 8085](#), [MOS 6502](#) (Commodore 64).

### (C) What is Mnemonics?

In microprocessors, **mnemonics are human-readable abbreviations for machine code instructions**, acting as memory aids (like [MOV](#) for Move, [ADD](#) for Addition) that make assembly language programming easier to understand and write than raw binary, with an assembler converting these symbolic codes into the 0s and 1s the processor executes. Each processor family (like x86, ARM) has its own specific set of mnemonics, representing operations such as data transfer, arithmetic, or jumps.

### Key aspects of mnemonics:

- **Memory Aid:** The word "mnemonic" comes from a Greek word meaning "mindful," helping programmers remember complex binary operations with simple letters.
- **[Assembly Language](#):** Mnemonics are the core of assembly language, a low-level programming language.
- **Operation Codes (Op-codes):** Each mnemonic corresponds to a specific [operation code](#) (opcode) that the CPU understands.
- **Examples:**
  - MOV AX, BX: Move data from the BX register to the AX register.
  - ADD A, B: Add the contents of register B to register A.
  - INR A: Increment the accumulator (register A) by one (for 8085 microprocessor).
- **Assembler's Role:** A program called an assembler translates these mnemonics (and their operands) into machine code (binary) before the processor can run the program.
- **Architecture Specific:** Mnemonics vary between processor architectures (e.g., Intel x86, ARM, MIPS), so an ADD instruction for one might be different for another.

## (D) Define: Assembly language, machine language, high level language

### Machine Language (Low-Level)

- **Definition:** The fundamental language of a computer, consisting of binary digits (0s and 1s) that the processor directly understands and executes.
- **Characteristics:** Hardware-dependent, extremely fast, non-portable, and very difficult for humans to read or write.
- **Translator:** None needed; it's the CPU's native code.

### Assembly Language (Mid-Level)

- **Definition:** A low-level language that uses symbolic codes (mnemonics) like ADD, SUB, MOV to represent machine instructions, making it slightly more readable than binary.
- **Characteristics:** Architecture-specific (e.g., x86, ARM), offers direct hardware control, but less portable than high-level languages.
- **Translator:** An **assembler** converts assembly code into machine code.

### High-Level Language (High-Level)

- **Definition:** Uses English-like keywords, familiar syntax, and abstractions (like loops, functions) for easier programming, like Python, C++, Java.
- **Characteristics:** Portable across different machines, easier to learn and use, slower execution than low-level languages due to translation overhead.
- **Translator:** A **compiler** (translates entire program) or **interpreter** (translates line-by-line) converts it to machine code.

## (E) What is an Assembler?

An assembler is a computer program that translates [assembly language](#), a low-level human-readable code using mnemonics (like ADD, MOV), into [machine code](#) (binary 0s and 1s) that a computer's processor can directly understand and execute, acting as a crucial translator for hardware-specific tasks. It enables direct interaction with hardware components like memory and registers, making it essential for performance-critical software and operating systems.

### How it works

- **Input:** Takes source code written in assembly language (e.g., LOAD R1, 100).
- **Process:** Replaces each mnemonic instruction and its operands with their corresponding binary machine code equivalent.

- **Output:** Produces object code (machine code) that the CPU executes, specific to the processor's architecture.

### Key characteristics

- **Hardware-dependent:** Assembly code and its generated machine code are tied to a specific CPU architecture.
- **Low-level:** Provides direct control over hardware, unlike high-level languages.

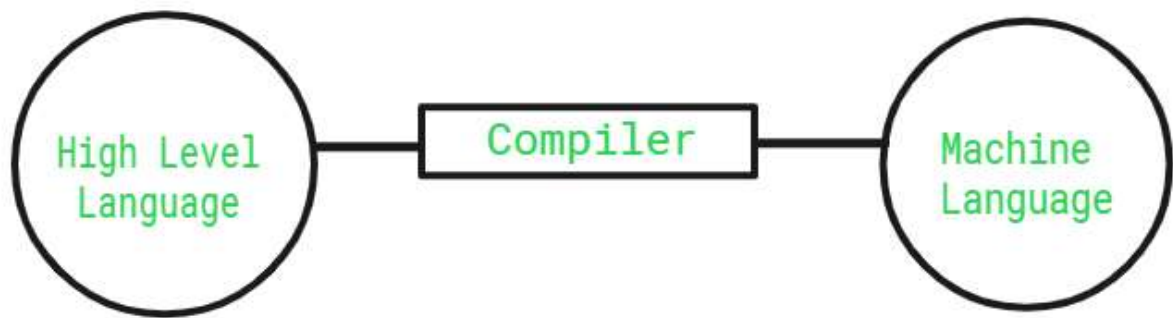
### Types of assemblers

- **Single-Pass Assembler:** Translates the code in one scan.
- **Multiple-Pass Assembler:** Requires several scans to resolve all instructions and symbols.
- **Self-Assembler:** Runs on a machine and produces code for that same machine (resident).
- **Cross-Assembler:** Runs on one machine but produces code for a different type of machine.

### (F) What is a Compiler?

A Compiler is a software that typically takes a high level language (Like C++ and Java) code as input and converts the input to a lower level language at once. It lists all the errors if the input code does not follow the rules of its language. This process is much faster than interpreter but it becomes difficult to debug all the errors together in a program.

A compiler is a translating program that translates the instructions of high level language to machine level language. A program which is input to the compiler is called a **Source program**. This program is now converted to a machine level language by a compiler is known as the **Object code**.



**(G) Which information given by address line?**

An address line provides crucial location details like [house/building numbers](#), [street names](#), [street suffixes](#), and secondary info like [apartment/suite numbers](#), guiding mail and deliveries to the exact physical spot, forming the core of a complete postal address.

**Key Information in Address Lines:**

- Address Line 1 (Primary): Contains the main building/house number, street name, and street type (e.g., St, Ave, Blvd).
  - *Example:* 123 Main Street
- Secondary Unit Designators (Often in Line 1 or Line 2): Apartment (Apt), Suite (Ste), Unit (Unit), Floor (Fl) numbers.
  - *Example:* Apt 4B or Suite 205
- Address Line 2 (Optional/Additional): Used for extra details, PO Boxes, or to clarify location in large complexes (e.g., Building D, Rear Entrance).
- Other Components (Often Separate Lines): City/Town, State/Province, Postal Code, Country.

**Why Address Lines Matter:**

- Precision: They break down a location into manageable parts for postal services and couriers.
- Delivery Success: Essential for avoiding misdeliveries, especially in multi-unit buildings or large campuses.
- Logical Structure: Keeps information organized, with primary details first, followed by finer points for delivery.

**(H) Data bus is always bidirectional True/False**

True

**(I) Microcontroller is faster than Microprocessor justify the statement.**

The statement that a **microcontroller is faster** can be justified in specific contexts—particularly regarding **system-level execution speed, real-time response, and startup time** for dedicated tasks.

Here is the justification for why a microcontroller can be faster:

- **On-Chip Integration (No External Delays):** A microcontroller (MCU) contains the CPU, RAM, ROM (flash memory), and peripherals on a single chip. Because these components are physically close, data does not need to travel across a circuit board to external RAM or memory, resulting in faster read/write times for instructions and data.
- **Faster Startup/Execution:**

Because of the on-chip memory, MCUs can load programs and initialize almost instantly

. Microprocessors (MPUs) must load operating systems from external, slower storage into RAM at startup, making them slower to initiate, especially in embedded applications

- **Efficient Task-Specific Processing:** MCUs are designed for specialized, single-purpose tasks (e.g., controlling a motor or sensor). They use Harvard architecture, which allows them to read instructions and access data memory simultaneously. This allows them to execute their specific code more efficiently than a general-purpose processor tackling complex, unrelated tasks.
- **Low Latency (Real-Time Control):** MCUs are ideal for real-time applications where a quick, predictable response is necessary. They can handle immediate I/O interrupts faster than an MPU, which may be bogged down by a complex operating system (like Linux or Windows).
- **Optimized Code Execution:** Because MCUs have limited memory, developers optimize code to be very compact, allowing for fast execution within the constraints of the chip.

| Feature      | Microcontroller (Faster Context)     | Microprocessor (Slower Context)                 |
|--------------|--------------------------------------|-------------------------------------------------|
| Memory       | Internal (On-chip), immediate access | External, requires time-consuming bus transfers |
| Startup      | Instant / Quick                      | Slow (needs to load OS)                         |
| Architecture | Harvard (Separate code/data)         | Von Neumann (Unified memory)                    |
| Best For     | Real-time, specific control          | High computation, multi-tasking                 |

## (J) Why do we need clock in microprocessor?

Clock speed, also known as clock rate or clock frequency, is a measure of how fast a computer's central processing unit (CPU) can execute instructions. It is typically measured in gigahertz (GHz). Higher clock speeds generally mean that a CPU can process more instructions per second, and thus can perform better on tasks that require fast processing.

Some advantages of high clock speed include:

1. **Improved performance:** As mentioned, a higher clock speed can result in faster processing and better performance on tasks that require a lot of computational power.
2. **Better multitasking:** A CPU with a high clock speed may be able to handle multiple tasks at once more efficiently, allowing for better multitasking.
3. **Greater efficiency:** In some cases, a CPU with a high clock speed may be able to complete tasks faster, which can lead to increased efficiency and productivity.

However, there are also some disadvantages to consider:

1. **Increased power consumption:** Higher clock speeds typically require more power, which can lead to increased energy consumption and heat production.
2. **Increased cost:** CPUs with high clock speeds may be more expensive than those with lower clock speeds.
3. **Limited by other factors:** A CPU's clock speed is just one factor that determines its performance. Other factors, such as the number of cores and the architecture of the CPU, can also impact a computer's overall performance.

It is usually measured in Megahertz or Gigahertz. In modern CPUs, we measure clock speed in Gigahertz.

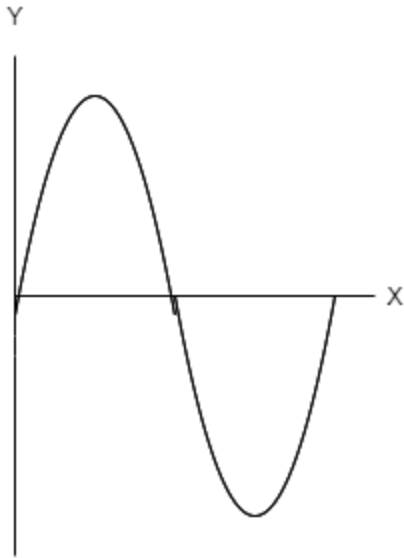
1 Megahertz = 1, 000, 000 cycles / sec

1 Gigahertz = 1, 000, 000, 000 cycles / sec

### Examples :

- A CPU with a clock speed of 4.2 MHz executes 4.2 Million cycles per second.
- A CPU with a clock speed of 4.2 GHz executes 4.2 Billion cycles per second.

As you can see, as clock speed increases, performance generally improves. However, there is a point at which increasing the clock speed no longer results in a significant increase in performance. This is known as the "performance plateau."



One Oscillation per second so 1 Hertz

|            |             |
|------------|-------------|
| Core Speed | 3592.72 MHZ |
| Multiplier | X 36.0      |
| Bus Speed  | 99.80 MHz   |

$$\text{Core Speed} = (\text{Bus Speed}) * (\text{Multiplier})$$

$$\text{Core Speed} = (99.80\text{MHz}) * (36.0) = 3592.8\text{MHz} = 3.50\text{GHz}$$

The speed of the CPU determines the number of calculations it can perform in one second of time. However, the speed of the CPU clock for processing power can be increased by 'overclocking'. But as you raise the clock speed there can be stability issues and your system might get crashed. Higher clock speeds can generate more heat so, to keep themselves away from dangerous overheating, processors will strangle down to a lower frequency when they get overheated. This can happen when you don't give enough power to your CPU. Overclocking may also lead to **overvolting**. Overvolting is when you raise the voltage given to the chip in order to increase the clock speed. Altair 8800, the first commercial PC used an Intel 8080 CPU with a clock rate of 2 MHz (2 Million cycles / sec) and the Intel P5 Pentium has a clock rate of 100 MHz (100 Million cycles / sec). In 2000, AMD was the first processor that reached 1 GHz (1 Billion cycles/sec). In 2002, an Intel Premium 4 was the first CPU with clock rate of 3GHz (3 Billion cycles/sec). In 2012 world record of highest clock speed is 8.794 GHz achieved by AMD FX-8350 using 2 cores. Highest base clock speed in production is the IBM zEC12, clocked at 5.5 GHz,

**(K) What are the advantages of an assembly language in comparison with high level language?**

**Assembly language** is a type of low-level programming language which is used for writing instructions for computers or other programmable devices. Since, it is a low-level language, therefore, it can communicate with computer hardware directly.

In assembly language, the computer codes are written using words and expressions that are easier to understand for human. The computer processor can only execute machine codes, hence it is required to convert the assembly codes into machine codes. For this purpose, a utility program is used to convert assembly code into executable machine code. This utility program which converts assembly code into machine code is called **assembler**.

The major advantages of assembly language are less memory requirement, less execution time, easier to write instructions and codes, etc.

**High-Level Language**, also called HLL, is a category of computer programming languages that use English like statements to write the computer instructions and codes. These are the most widely used programming languages because they are easy to understand to human being. However, similar to the assembly language, the CPU cannot process the high-level language codes directly, i.e. they need to be translated first into the executable machine codes. For this, there are two types of language translators used namely, interpreter and compiler.

The major advantages of high-level languages include easy to write, debug, and understand, machine independent, etc. The common examples of high-level languages are C, C++, Java, Python, C#, etc.

| <b>Parameter</b> | <b>Assembly Language</b>                                                                                                | <b>High-Level Language</b>                                                                                   |
|------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Definition       | Assembly language is a computer programming language in which abbreviated keywords are used to write instruction codes. | High-level language is a computer program language in which English like statements are used to write codes. |

|                         |                                                                                                                                                                                                 |                                                                                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Language translator     | A language translator called "assembler" is required to convert the assembly language code into the machine code.                                                                               | High-level language requires an interpreter or a compiler to convert the high-level language codes into the executable machine codes. |
| Level of language       | Assembly language is a low-level language.                                                                                                                                                      | High-level language, as the name implies, is high-level language.                                                                     |
| Programmer friendliness | Assembly language is less programmer friendly programming language.                                                                                                                             | High-level language is highly user friendly programming language.                                                                     |
| Speed of execution      | Computer instructions written in assembly language execute faster.                                                                                                                              | Computer instructions written in high-level language execute slower.                                                                  |
| Machine dependency      | Assembly language is machine dependent.                                                                                                                                                         | High-level language is machine independent.                                                                                           |
| Prone to error          | Assembly language is more prone to errors.                                                                                                                                                      | The chances of errors in high-level languages are less.                                                                               |
| Memory requirement      | Assembly language codes require less memory space.                                                                                                                                              | High-level language codes require more memory space.                                                                                  |
| Code length             | The length of executable codes in assembly language is shorter.                                                                                                                                 | The length of executable codes in high-level language is longer.                                                                      |
| Debug                   | Assembly language codes are relatively difficult to debug. It is more challenging and time- consuming.                                                                                          | High-level language codes are very easy to debug.                                                                                     |
| Complexity              | Assembly language is a complex programming language, as to write the instruction codes in assembly language, the programmer must have a deep understanding of hardware and system architecture. | High-level languages are easy to write codes without much knowledge about the computer hardware and architecture.                     |
| Efficiency              | Assembly language codes are more efficient.                                                                                                                                                     | High-level language codes are less efficient, as the coder has less control over the underlying hardware.                             |
| Readability             | The readability of assembly language codes is less.                                                                                                                                             | High-level language codes are more readable.                                                                                          |

|                           |                                                                                                                     |                                                                                                   |
|---------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Development time & effort | Assembly language programs take more time and effort to develop.                                                    | High-level language programs require less development time and effort.                            |
| Memory management         | Assembly language codes require manual memory management.                                                           | High-level language codes provide automatic memory management.                                    |
| Syntax                    | Assembly language uses symbolic representation of machine codes.                                                    | High-level languages use keywords and statements similar to English language.                     |
| Applications              | Assembly language is primarily used to program processors, microcontrollers, embedded systems, device drivers, etc. | High-level languages are mainly used for developing software applications, web applications, etc. |

**(L) Write full name of: RAM, ROM, ALU, SSI, MSI, VLSI, and ASCII.**

- **RAM:** Random Access Memory.
- **ROM:** Read Only Memory.
- **ALU:** Arithmetic Logic Unit.
- **SSI:** Small Scale Integration (referring to integrated circuits with few transistors).
- **MSI:** Medium Scale Integration (more transistors than SSI, implementing basic functions).
- **VLSI:** Very Large Scale Integration (combining millions/billions of transistors, like microprocessors).
- **ASCII:** American Standard Code for Information Interchange (a character encoding standard).

**(M) What is bus?**

A **bus** is a core communication system, essentially a shared pathway (wires/traces) on the motherboard, that transfers data, addresses, and control signals between computer components like the CPU, memory, and peripherals, acting like a digital highway to manage information flow efficiently. These buses ([Data Bus](#), [Address Bus](#), [Control Bus](#)) allow the CPU to talk to memory and I/O devices, forming the backbone of internal computer communication.

## **(N) What do mean by peripherals?**

A peripheral device is any external device connected to a computer that adds functionality but is not part of the core computer system. These devices typically handle input, output, or both, and are also known as input-output (I/O) devices. While not essential for a computer to perform its basic operations, peripherals enhance the user's experience by expanding the system's capabilities. Common examples include keyboards, mice, printers, and external drives. Although the term is sometimes used loosely, it generally refers to devices located outside the computer case.

### **Classification of Peripheral Devices**

It is generally classified into four basic categories which are given below:

#### **1. Input Devices:**

An input device is defined as a device that converts incoming data and instructions into a pattern of electrical signals in binary code that are comprehensible to a digital computer.

- **Keyboard:** A keyboard is an input device that allows users to enter text and commands into a computer system.
- **Mouse:** A mouse is an input device that allows users to control the cursor on a computer screen.
- **Scanner:** A scanner is an input device that allows users to convert physical documents and images into digital files.
- **Microphone:** A microphone is an input device that allows users to record audio.

#### **2. Output Devices:**

An output device is generally the reverse of the input process and generally translates the digitized signals into a form intelligible to the user. The output device is also performed for sending data from one computer system to another. For some time punched card and paper tape readers were extensively used for input, but these have now been replaced by more efficient devices. Example:

- **Monitor:** It is an output device that displays visual information from a computer system.
- **Printer:** It is an output device that produces physical copies of documents or images.
- **Speaker:** It is an output device that produces audio.

### 3. Storage Devices:

Storage devices are used to store data in the system which is required for performing any operation in the system. The storage device is one of the most required devices and also provides better compatibility. Example:

- **Hard Drive:** A hard drive is a storage device that stores data and files on a computer system.
- **USB Drive:** A USB drive is a small, portable storage device that connects to a computer system to provide additional storage space.
- **Memory Card:** A memory card is a small, portable storage device that is commonly used in digital cameras and smartphones.
- **External Hard Drive:** An external hard drive is a storage device that connects to a computer system to provide additional storage space.

### 4. Communication Devices:

Communication devices are hardware devices that enables data exchange between computer systems or networks. These include:

- **Modem:** A modem is a communication device that allows a computer system to connect to the internet.
- **Network Card:** A network card is a communication device that allows a computer system to connect to a network.
- **Router:** A router is a communication device that allows multiple devices to connect to a network.

### Importance of Peripheral Devices

Peripheral devices are important for enhancing functionality of a computer. Here's why peripheral devices are so important:

- **Enhanced Functionality:** Peripheral devices expand a computer's capabilities, enabling it to perform various tasks.
- **User Interaction:** Input devices like keyboards and mice allow users to control and interact with the system.
- **Data Output:** Output devices such as monitors and printers display or present processed data.
- **Data Storage:** Storage devices ensure important data is saved and can be accessed or backed up when needed.

- **Communication:** Communication devices enable network connectivity, allowing the computer to communicate with other systems or networks.
- **Increased Efficiency:** Peripherals improve the overall efficiency and usability of the computer by adding specialized functions.

### Peripheral Devices Connecting Options

Peripheral devices can connect to a computer through various methods. Each offers different speeds, ranges, and compatibility. Common connecting options include:

- **USB:** [USB](#) is common for keyboards, mice, printers, and external storage devices; supports both wired and wireless connections.
- **Bluetooth:** [Bluetooth](#) is wireless connection for devices like headphones, speakers, and mice.
- **Wi-Fi:** [Wi-Fi](#) is used for wireless networked devices like printers and cameras.
- **HDMI:** [HDMI](#) connects monitors, projectors, and TVs for high-definition video and audio.
- **Ethernet:** Wired connection for network communication, often for internet or [LAN](#) access.
- **Thunderbolt:** High-speed connection for external devices like storage drives and monitors.
- **VGA:** [VGA](#) is older connection for monitors or projectors, supports lower video resolutions.
- **DisplayPort:** Similar to HDMI, used for connecting high-definition monitors.

### (O) How much memory access by 8085 microprocessor.

The 8085 microprocessor can directly access **64 kilobytes (KB) of memory**. This capacity is determined by its **16-bit address bus**.

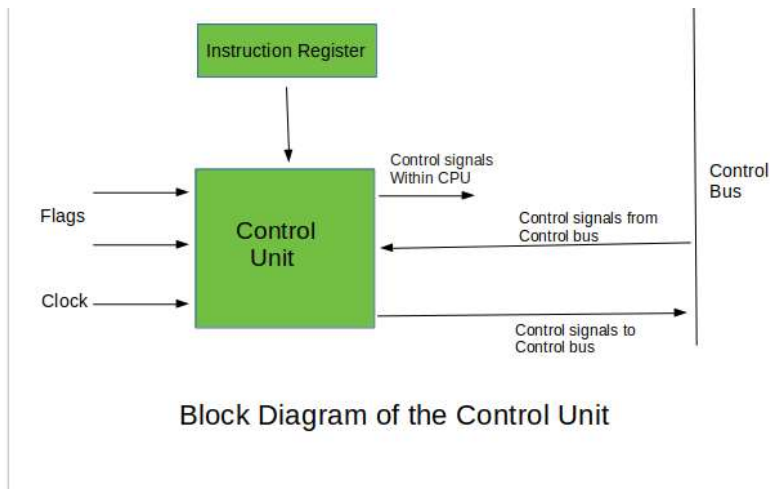
### (P) ALU stand for

ALU primarily stands for [Arithmetic Logic Unit](#)

### (Q) What is purpose of Control Unit of Microprocessor?

The control unit is the part of the computer's central processing unit (CPU) which directs the operation of the processor. It fetches instructions from memory, decodes them, and generates control signals to manage the ALU, memory, and I/O devices.

- Its role is to coordinate instruction execution, and its functions vary depending on CPU architecture.
- Devices that need a Control Unit (CU) include CPUs and GPUs.



### Functions of the Control Unit

- It coordinates the sequence of data movements into, out of, and between a processor's many sub-units.
- It interprets instructions.
- It controls data flow inside the processor.
- It receives external instructions or commands, which it converts to a sequence of control signals.
- It controls many execution units (i.e., [ALU](#), data buffers and [registers](#)) contained within a CPU.
- It also handles multiple tasks, such as fetching, decoding, execution handling and storing results.

The control unit of a CPU fetches and executes instructions, playing a critical role in system performance. Its Design ensures smooth operation of various components.

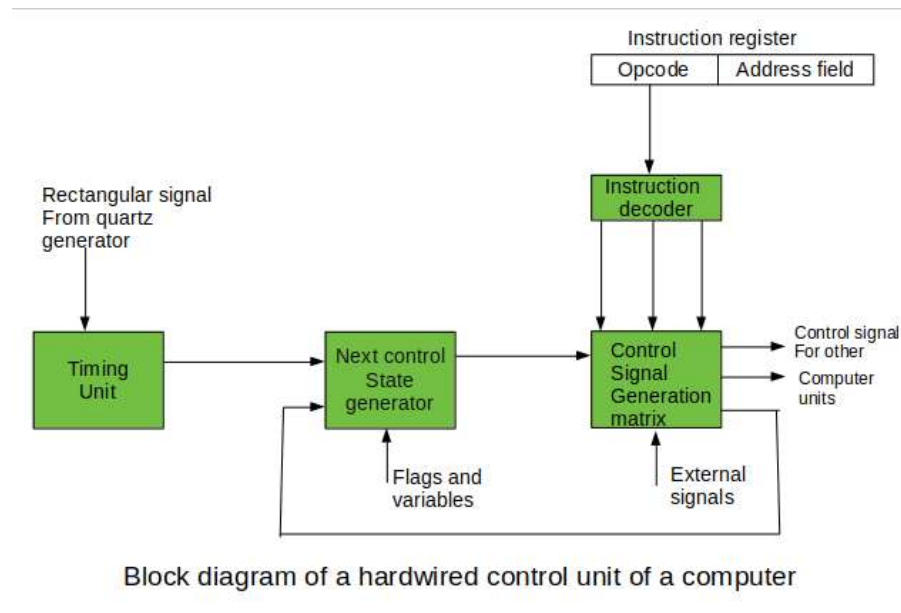
### Types of Control Unit

There are two types of control units:

- Hardwired
- Micro programmable control unit.

### Hardwired Control Unit

In a Hardwired Control Unit, fixed hardware logic generates control signals based on the instruction's opcode. The opcode is decoded, and the decoder activates lines that feed a control signal generator matrix. This matrix, similar to a programmable logic array, combines decoded signals, control states, and external inputs to produce the required execution signals.



- Control signals must be generated throughout the instruction execution cycle, not at a single point. Accordingly, the control unit organizes a sequence of states, with some control signals fed back to the next state generator matrix.
- The timing unit (driven by a quartz generator) provides timing signals. When a new instruction arrives, the control unit begins fetching, moves through execution states, and responds to changes in timing, flags, or interrupts by shifting states.
- External signals such as interrupts trigger dedicated states for handling them. Flags and state variables guide the choice of states during execution.
- The final states of the cycle begin fetching the next instruction, and if a stop instruction is encountered, the control unit enters an OS state, waiting for the next command.

### Micro Programmable control unit

In [microprogrammed control units](#), subsequent instruction words are fetched into the instruction register in a normal way. However, the operation code of each instruction is not directly decoded to enable immediate control signal generation but it comprises the initial address of a microprogram contained in the control store.

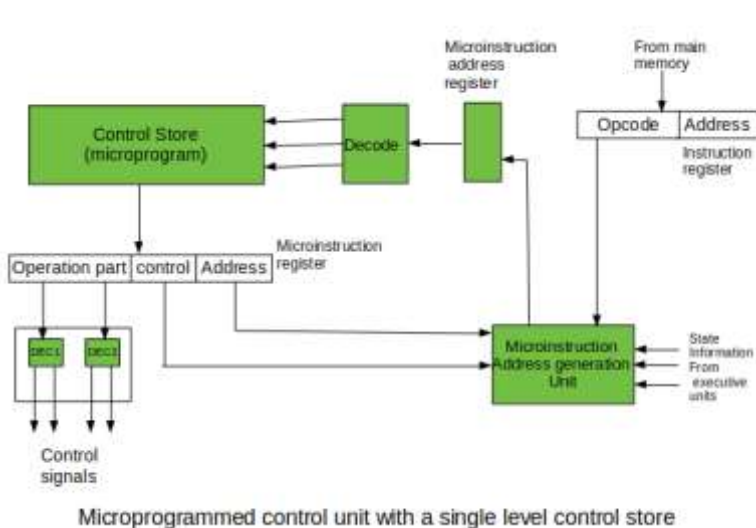
### Some Important Terms

1. **Control Word:** A control word is a word whose individual bits represent various control signals.

2. **Micro-routine:** A sequence of control words corresponding to the control sequence of a machine instruction constitutes the micro-routine for that instruction.
3. **Micro-instruction:** Individual control words in this micro-routine are referred to as microinstructions.
4. **Micro-program:** A sequence of micro-instructions is called a micro-program, which is stored in a ROM or RAM called a Control Memory (CM).
5. **Control Store:** the micro-routines for all instructions in the instruction set of a computer are stored in a special memory called the Control Store.

### Single-Level Control Store

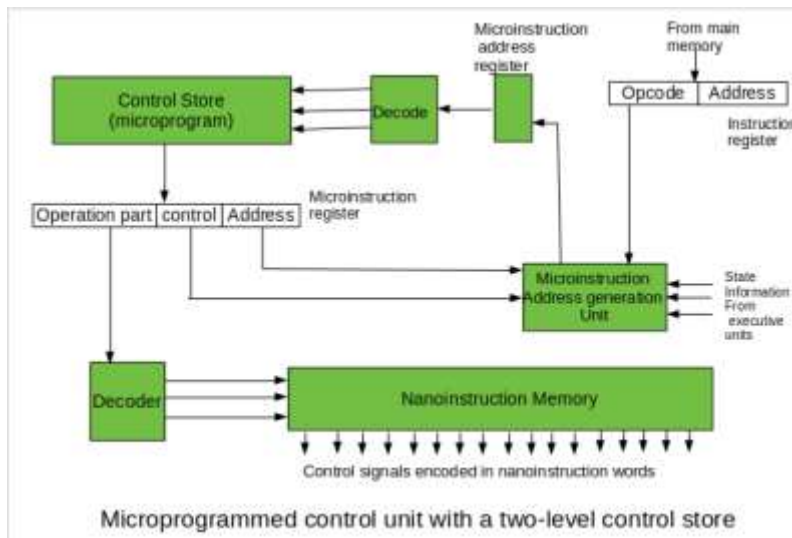
The opcode from the instruction register points to the address in the control store where the microprogram starts. The first microinstruction is loaded, containing encoded control signals and the address for the next microinstruction. Microinstruction fields are decoded, and the control field determines which [addressing mode](#) or operation is used.



Conditional addressing applies processor flags to select the next microinstruction. The final microinstruction fetches the next instruction from memory.

### Two-Level Control Store

In a two-level control unit, microinstructions point to nano-instructions instead of directly containing control signals. The microinstruction's operation field holds the address of a nano-instruction, which includes the actual encoded control signals. The nano-instruction memory stores all needed control signal combinations for executing the full instruction set, written once and reused across microprograms.



In this structure, microinstructions are shorter, reducing the total control memory size. Control sequencing is handled by microinstructions, while control signals are generated from nanoinstructions, often in simple 1-bit/1-signal encoding without further decoding.

### (R) Justify “ Microprocessor is a programmable device”

microprocessor is considered a programmable device because it is an integrated circuit designed to accept, interpret, and execute a set of binary instructions (a program) to perform a wide variety of tasks without needing physical reconfiguration.

Justification for this statement is based on the following functional characteristics:

- **Instruction Set Execution:** A microprocessor has a pre-designed "instruction set" (like RISC or CISC) that dictates what operations it can perform, such as addition, subtraction, logical AND/OR, and data movement.
- **Fetch-Decode-Execute Cycle:** The device operates by constantly fetching binary instructions (machine code) from memory, decoding them to determine the required action, and executing those commands.
- **Flexibility and Versatility:** Unlike fixed-function hardware, the same microprocessor chip can function as a calculator, a traffic light controller, or a component in a computer, depending entirely on the software (program) loaded into the memory.
- **Programmable Registers:** Microprocessors contain internal registers (like the Program Counter and Accumulator) that are accessible to the user, allowing for data manipulation and control over the flow of the program.
- **Decision-Making Capability:** Through branch and jump instructions, a microprocessor can make logical decisions based on results, enabling complex, conditional programming.

### **(S) Justify “The direction of address bus is unidirectional”**

The address bus is unidirectional because it serves a single purpose: to carry memory addresses from the microprocessor (CPU) to memory or I/O devices, specifying where to read or write, with the CPU always being the source of addresses, not a destination for them. In contrast, the [data bus](#) is [bidirectional](#) because it needs to carry data both ways (CPU to memory for writing, memory to CPU for reading).

#### **Why the CPU Sends Addresses (Unidirectional):**

- **Source of Truth:**

The CPU is the master component that dictates which memory location or device to interact with.

- **Location Identification:**

Addresses are identifiers for storage spots; they don't contain data themselves but point to where data lives.

- **No External Addressing:**

Other components don't "address" the CPU for memory access; they respond to the CPU's requests.

#### **Why Data Travels Both Ways (Bidirectional):**

- **Read Operation:**

CPU sends an address, then waits for data to flow back from memory/I/O on the data bus.

- **Write Operation:**

CPU sends an address and then sends the data out to that location on the data bus.

### **(T) What is difference between 4 bit microprocessor and 8 bit microprocessor?**

#### **[4-Bit Microprocessors](#) (e.g., Intel 4004)**

- **Data Width:** 4 bits (one nibble).
- **Data Capacity:** Can represent values from 0 to 15 (

$$2^4 - 1$$

24-1

).

- **Performance:** Slower and less powerful; suitable for basic arithmetic and control.
- **Applications:** Early calculators, simple embedded systems, and novelty devices.

#### 8-Bit Microprocessors (e.g., Intel 8080, Zilog Z80)

- **Data Width:** 8 bits (one byte).
- **Data Capacity:** Can represent values from 0 to 255 (

$$2^8 - 1$$

28-1

).

- **Performance:** More capable, can perform arithmetic and logic on larger numbers, and handle more complex instructions.
- **Applications:** Early personal computers, gaming consoles (Atari), and more advanced embedded systems.

#### **(U) What is difference between assembly language and machine language?**

| Machine Language                                                                                                      | Assembly Language                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Machine language is only understood by the computers.                                                                 | Assembly language is only understood by human beings not by the computers.                              |
| In machine language data is only represented with the help of binary format (0s and 1s), hexadecimal and octadecimal. | In assembly language data can be represented with the help of mnemonics such as Mov, Add, Sub, End etc. |
| Machine language is very difficult to understand by the human beings.                                                 | Assembly language is easy to understand by the human being as compared to machine language.             |
| Modifications and error fixing cannot be done in machine language.                                                    | Modifications and error fixing can be done in assembly language.                                        |
| Machine language is very difficult to memorize so it is not possible to learn the machine language.                   | Easy to memorize the assembly language because some alphabets and mnemonics are used.                   |

|                                                                                             |                                                                                        |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Execution is fast in machine language because all data is already present in binary format. | Execution is slow as compared to machine language.                                     |
| There is no need of translator. The machine understandable form is the machine language.    | Assembler is used as translator to convert mnemonics into machine understandable form. |
| Machine language is hardware dependent.                                                     | Assembly language is the machine dependent and it is not portable.                     |

#### **(V) What is use of Registers in microprocessor?**

**Accumulator (A) register:** The accumulator register is the most commonly used register in the 8085 microprocessor. It is used for arithmetic and logical operations, as well as for input/output (I/O) operations. The accumulator is also used as a temporary storage location for data.

**Program Counter (PC) register:** The PC register is used to keep track of the memory location of the current instruction. When an instruction is executed, the PC register is automatically incremented to point to the next instruction in memory.

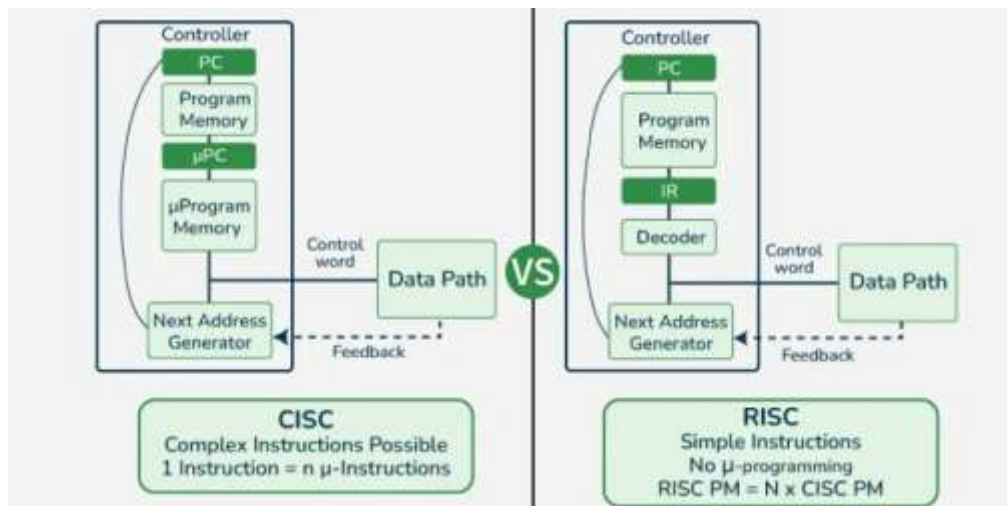
**Stack Pointer (SP) register:** The SP register is used to keep track of the top of the stack. The stack is used for temporary storage of data and return addresses during subroutine calls.

**Flag Register:** The flag register is used to store status information about the results of arithmetic and logical operations, including whether a result is negative, zero, or carry.

**General Purpose Registers (B, C, D, E, H, and L):** These registers are used for general purpose storage of data, as well as for addressing memory locations. They can be used in pairs as 16-bit registers, such as BC, DE, and HL, for more efficient addressing of memory locations.

Instruction Register (IR) and Machine Cycle Register (MCR): These registers are used internally by the microprocessor to decode instructions and control the timing of machine cycles.

**(W) What stand for CISC and RISC?**



**Reduced Instruction Set Architecture (RISC)**

RISC simplifies processor design by using a small, uniform set of instructions. Each instruction performs a basic operation (e.g., load, compute, store) and is designed to execute in a single clock cycle, enabling efficient pipelining and simpler hardware.

**Characteristics of RISC**

- Simpler instruction, hence simple instruction decoding.
- Instruction comes in the form of one word.
- An instruction takes a single clock cycle to get executed.
- More general-purpose registers for register-to-register operations.
- Simple Addressing Modes.
- Optimized for pipelining due to uniform instruction size and simplicity.

**Complex Instruction Set Architecture (CISC)**

CISC reduces the number of instructions a program needs by using a large set of complex, variable-length instructions. A single instruction can perform multiple operations (e.g., load, compute, and store), which may take multiple clock cycles.

## Characteristics of CISC

- Complex instruction, hence complex instruction decoding.
- Instructions are larger than one-word size.
- Instruction may take more than a single clock cycle to get executed.
- Less number of general-purpose registers as operations get performed in memory itself.
- Complex Addressing Modes.

## CPU Performance of RISC and CISC

Both approaches try to increase the CPU performance

$$CPU\ Time = \frac{Seconds}{Program} = \frac{Instructions}{Program} \times \frac{Cycles}{Instructions} \times \frac{Seconds}{Cycle}$$

CPU Time

- **RISC:** Reduce the cycles per instruction at the cost of the number of instructions per program.
- **CISC:** The CISC approach attempts to minimize the number of instructions per program but at the cost of an increase in the number of cycles per instruction.

## (X) What is Op-code?

An [opcode](#) (operation code) is the part of a machine language instruction that tells a computer's CPU what specific action to perform, like adding, moving data, or jumping; it's the command within a larger instruction that specifies the task, while [operands](#) provide the data or memory locations the operation will use, forming complete instructions like "ADD R1, R2" where "ADD" is the opcode and "R1, R2" are operands. Opcodes are fundamental to how hardware executes software, with different processors having unique sets of them.

## Key aspects of opcodes:

- **Function:** Defines the operation (e.g., ADD, SUB, MOV, JMP).
- **Format:** Often the first part of a binary machine instruction, followed by operands.
- **Context:**

Essential for CPUs and ALUs (Arithmetic Logic Units) to interpret instructions.

- **Assembly Language:** Represented by mnemonics (like ADD, MOV) that translate directly to binary opcodes.

- **Example:** In ADD A, B, the ADD is the opcode, telling the CPU to add, while A and B are operands (registers) containing the data to be added.

#### Opcodes in different contexts:

- **Microprocessors:** A core concept for understanding how CPUs execute commands.
- **Malware Analysis:** Analyzing opcode sequences helps detect malicious software behavior.
- **Web3/Blockchain:** In virtual machines (like Ethereum's EVM), opcodes are fundamental building blocks for smart contracts.

### (Y) What is Operand?

In a microprocessor, an **operand** is the data or the memory location/register that an instruction (the opcode) acts upon, essentially answering "what" to operate on. While the opcode (e.g., ADD, MOV) tells the CPU *what* to do (like add or move), the operand specifies *where* that data is or *what* the data value is, such as a register (R1, AX), a memory address, or a direct number.

#### Key Aspects of Operands:

- **Data/Location:** It's the value, memory location, or register holding the value that the operation will use or modify.
- **Part of an Instruction:**

Every instruction is typically composed of an opcode and one or more operands

- **Examples in Assembly:**
  - In ADD R1, R2, R1 and R2 are operands (registers) for the ADD operation.
  - In MOV BX, AX, AX (source) and BX (destination) are operands for the MOV (move) instruction.
- **Types:** Operands can be direct numbers (literals), contents of registers, memory addresses, or even implicit (understood by the opcode).

#### Analogy:

Think of a recipe:

- **Opcode:** "Chop" or "Mix" (the action)
- **Operand:** "Onions" or "Flour" (the ingredients/data being acted upon).

In short, the operand provides the necessary data or address for the microprocessor's [ALU](#) (Arithmetic Logic Unit) to perform the task defined by the opcode.

### **(Z) What is difference between general purpose microprocessor and special purpose microprocessor?**

General-purpose processors (GPPs) are flexible for diverse tasks (like CPUs in PCs) with broad instruction sets, while special-purpose processors (SPPs) are highly optimized for specific functions (like [DSPs](#) for audio, [GPUs](#) for graphics) using dedicated architectures (like Harvard) for maximum efficiency, performance, and lower power in their niche, though with less adaptability. Key differences lie in **flexibility vs. efficiency**, architecture (Von Neumann vs. Harvard), instruction sets, and application focus, with GPPs for general computing and SPPs for specialized signal/data processing.

#### **General-Purpose Processors (GPPs)**

- **Purpose:** Run diverse applications, operating systems (e.g., Windows, Linux).
- **Architecture:**

Typically Von Neumann (shared memory for data/instructions).

- **Flexibility:** High; programmable for many tasks.
- **Performance:** Good for general computing, less efficient for intensive, repetitive tasks.
- **Examples:** Intel Core i7, AMD Ryzen, [ARM Cortex-A](#) series.

#### **Special-Purpose Processors (SPPs)**

- **Purpose:** Excel at specific tasks like audio/video processing, graphics, control.
- **Architecture:** Often Harvard (separate memory for data/instructions) for speed.
- **Flexibility:** Low; optimized for a narrow range of functions.
- **Performance:** High for their specific job (e.g., fast multiplication in [DSPs](#)).
- **Examples:** Digital Signal Processors (DSPs), [Graphics Processing Units](#) (GPUs).

#### **Key Distinctions**

- **Efficiency vs. Versatility:** GPPs are versatile but less efficient for single tasks; SPPs are highly efficient but limited in scope.
- **Architecture:** GPPs often use Von Neumann; SPPs (like DSPs) often use Harvard for parallel data/instruction fetching.
- **Instruction Set:** GPPs have complex, varied instructions; SPPs have specialized instructions (e.g., Multiply-Accumulate) for their domain.